

Development Plan

Software Engineering

Team #14, Troitsky Toolbox
Pedro Azevedo
Mason Azzopardi
Michael Roberts
Jack Easton

Table 1: Revision History

Date	Developer(s)	Change
Date1	Name(s)	Description of changes
Date2	Name(s)	Description of changes
...	...	...

[Put your introductory blurb here. Often the blurb is a brief roadmap of what is contained in the report. —SS]

[Additional information on the development plan can be found in the [lecture slides](#). —SS]

1 Confidential Information?

[State whether your project has confidential information from industry, or not. If there is confidential information, point to the agreement you have in place. —SS]

[For most teams this section will just state that there is no confidential information to protect. —SS]

2 IP to Protect

[State whether there is IP to protect. If there is, point to the agreement. All students who are working on a project that requires an IP agreement are also required to sign the “Intellectual Property Guide Acknowledgement.” —SS]

3 Copyright License

[What copyright license is your team adopting. Point to the license in your repo. —SS]

4 Team Meeting Plan

[This section is for meetings between the team members, and meetings between the team and their supervisor/advisor (as appropriate). —SS]

[How often will the team meet? where? when? —SS]

[If the meeting is a physical location (not virtual), out of an abundance of caution for safety reasons you shouldn’t put the location online —SS]

[To help schedule meetings, you can use the lecture/tutorial time when we do not have anything else scheduled. —SS]

[How often will you meet with your supervisor/advisor? when? where? —SS]

[Will meetings be virtual? At least some meetings should likely be in-person. —SS]

[How will the meetings be structured? There should be a chair for all meetings. There should be an agenda for all meetings. —SS]

5 Team Communication Plan

[Issues on GitHub should be part of your communication plan. —SS]

6 Team Member Roles

6.1 Role Descriptions

1. **Manager:** The manager will serve as a coordinator between the other members, ensuring that communication is constant and issues are resolved in time and amicably. The manager will also serve as the standard meeting lead, with room for this to change as meeting topics become specified.
2. **Notetaker:** The notetaker will be responsible for taking meeting minutes in supervisor, TA and general team meetings, along with taking notes during lectures for the team. This role will focus on maintaining a standardized note structure and level of detail throughout sessions.
3. **Troitsky Club Communication Associate:** This role will be responsible for maintaining constant communication with the Troitsky club, scheduling interviews, user tests, and any other interaction the team will have with the club. This will become a more generalized role as the team becomes more integrated into the club.
4. **Reviewer/Administrator:** The reviewer/administrator role will be responsible for final reviews of PRs and ensuring that submissions are complete by the set deadlines. While the team as a whole should be integrated into this process, the administrator will serve as the final check and insurance that items will be complete on time.

6.2 Role Assignments

The assignment structure of the team will be chosen initially based on prior experience and general inclination to each role. The reassignment of these roles will be completed between submissions, focusing on allowing each member to take on the given roles. In cases of changes and new information arising, the role assignments and structure may be adjusted through communication with all members.

The roles listed below outline the base for team meeting operations, standardizing key roles

- **Manager/Team Leader** The manager will be tasked with organizing the team, ensuring that cooperation as a whole is kept in check.
- **Notetaker** The notetaker will be responsible for taking either meeting minutes during team/supervisor meetings or, when applicable, taking lecture notes for the benefit of the team.
- **Troitsky Club Communication Organizer**
- **Reviewer/Administrator**

7 Workflow Plan

- How will you be using git, including branches, pull request, etc.?
- How will you be managing issues, including template issues, issue classification, etc.?
- Use of CI/CD

8 Project Decomposition and Scheduling

- How will you be using GitHub projects?
- Include a link to your GitHub project

[How will the project be scheduled? This is the big picture schedule, not details. You will need to reproduce information that is in the course outline for deadlines. —SS]

9 Proof of Concept Demonstration Plan

Due to the nature of the Toolbox providing a variety of services and features, the plan for the Proof of Concept Demonstration has a unique way it approaches the risks presented in the project.

9.1 Risks

Since many tools operate independently from each other, or in small groups, the risks presented by a single tool's performance/attainability do not directly impact the attainability of another.

With that being said, there is a group of tools and features that have a high level of risk associated with them due to their complexity and connectivity to each other. These related and complex tools would be uniquely tailored to assist the McMaster Troitsky with visualization, performance estimation, and rule compliance testing of bridge designs.

Since the specific details of rule compliance and performance estimation aren't relevant for the Proof of Concept Demo, the focus will be on extracting and visualizing data from the 3D models of a given bridge design. The plan to demonstrate the concept of functionality is possible by loading a STEP file (.stp – universal 3D CAD file format) and displaying a visualization or rendering of the file's contents. Along with the visualization, numerical data will be extracted from the .stp file related to the bridge, which eventually will be used to support the competition rule compliance assessment tool and projected bridge performance and metrics tool. This is the individually most complicated feature that will be implemented in the Troitsky Toolbox, so it serves as one of the greatest risks to the project.

It is also worth noting that although there are many different tools that will need to be appropriately connected within the program, it is not a large risk or feasible to demonstrate during the Proof of Concept Demo.

9.2 Risk Mitigation

Due to the nature of the Toolbox, risk mitigation in the specific project context is also unique. Since the biggest isolated risk comes from an individual tool, the biggest negative impact of that risk (if not successfully demonstrated) is that the Toolbox cannot provide that specific feature. For this project, a "failed" POC demo would mean that there are a few features (compared to the many planned to implement) that aren't achievable within the scope of the project. This failure means that the team would modify the scope of the project to include an alternate related feature that bypasses the technical limitation or refine, improve, iterate, and add to alternate functionality of the Toolbox.

10 Expected Technology

[What programming language or languages do you expect to use? What external libraries? What frameworks? What technologies. Are there major components of the implementation that you expect you will implement, despite the existence of libraries that provide the required functionality. For projects with machine learning, will you use pre-trained models, or be training your own model? —SS]

[The implementation decisions can, and likely will, change over the course of the project. The initial documentation should be written in an abstract way; it should be agnostic of the implementation choices, unless the implementation choices are project constraints. However, recording our initial thoughts on implementation helps understand the challenge level and feasibility of a project. It may also help with early identification of areas where project members will need to augment their training. —SS]

Topics to discuss include the following:

- Specific programming language
- Specific libraries
- Pre-trained models
- Specific linter tool (if appropriate)
- Specific unit testing framework
- Investigation of code coverage measuring tools
- Specific plans for Continuous Integration (CI), or an explanation that CI is not being done
- Specific performance measuring tools (like Valgrind), if appropriate

- Tools you will likely be using?

[git, GitHub and GitHub projects should be part of your technology. —SS]

This project shall be accessible via browser for desktop users. Data shall be stored in a robust database. The back-end logic shall be rewritten a powerful and easily maintainable programming language, preferably with a utility which will aid front-end development. The current likely option is the .NET programming suite, backed by a Microsoft SQL Server database, with the back-end in C# and the front-end utilizing Razor (C# HTML) pages to facilitate rendering web pages. A great advantage of doing so is its language integrated query (LINQ) suite, which transfers specialized, elegantly-written code into an efficient SQL query.

Beside standard libraries, state-of-the-art structural engineering utilities will likely be used to aid calculations, for implementing complex calculations is prone to error, especially for developers not well-versed in civil engineering. A likely option is the AutoCAD application programming interface (API) with the Inventor API of AutoDesk Inventor being an alternate.

Linting shall be used to ensure production code is free of syntax errors, style issues and bad practices. One option is [CS Lint](#) by Lucas Pimentel, freely available as it is licensed under the MIT license.

A unit testing framework shall be implemented to ensure correctness, like the native MS Test, NUnit or xUnit. The comprehensiveness of the unit tests shall be measured in part by code coverage, like the Coverlet tool of XUnit. Another tactic to increase comprehensiveness could be to correlate the projected frequency of execution of a path with the number of unit tests it has. Such frequency would be determined through consultation with this project's users.

Continuous integration shall be implemented through a distributed version-control system, specifically git on a GitHub-hosted repository. Each change to production shall be proposed by its developer through a merge request from a feature branch which is exclusively theirs to the production branch. Barring an extraordinary emergency, the change must be approved by another developer before merging, preferably one who is working on a feature which closely interacts with the proposed change.

11 Use of AI and LLMs

[This section is a continuation of the previous section. The role of LLMs in your project as important enough to warrant their own section. How will your team be using LLMs for code and documentation development? What tools will you be using? How will you verify the output of your LLMs? —SS]

This project is not a machine learning project, as it will implement many subordinate services for the Troitsky club. However, utilization of robust AI chatbots (e.g. Claude Code, ChatGPT, Copilot) to write code will increase developer efficiency by decreasing the time each developer needs to spend drafting and revising code. However, AI is prone to hallucinations and other vulnerabilities. Therefore, each developer shall thoroughly review all AI-generated code

they generate before creating a pull request and not submit the code for a pull request until they completely understand the code and are convinced of its validity.

12 Coding Standard

[What coding standard will you adopt? —SS]

This project shall adopt the standard .NET coding conventions, available [here](#).

LINQ shall be utilized unless the equivalent raw SQL query provides a significant performance boost.

13 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

[This section is about the use of AI to write this document, not your planned use of AI for your project. You discuss that topic in one of the sections in the body of this report. —SS]

13.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

13.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

13.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

Appendix — Reflection

1. Why is it important to create a development plan prior to starting the project?
2. In your opinion, what are the advantages and disadvantages of using CI/CD?
3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

Pedro Azevedo

1. The creation of the development plan prior to starting the project is key to establishing the patterns and strategies that the team will use to create clean and maintainable code. Part of the value of this project is its ability to stand without anyone from the current group being a part of it, and central to that will be having well-documented code. The development plan also made us settle on some key factors of the project that may have been decided more on a whim if it wasn't set in stone.
2. I believe the main benefit of CI/CD is the standardization of builds and submissions, ensuring that what we work on will be properly replicated when built by anyone else, including the TAs and instructors. It ensures that the main branch is always a working version of the service, one that has passed all the checks set up by the team. It brings peace of mind when the project gets busier, and development is moving quicker, ensuring no changes that break the project are made.
3. There weren't any major disagreements during this deliverable, just different suggestions for technologies for the development plan, which were all decided through some more thorough research. When certain decisions relied on some technical knowledge, the team would often turn to the member who best understood that topic.

Appendix — Team Charter

External Goals

1. The primary external goals related to the benefit and success of this product in improving McMaster's Troitsky teams; fundamentally, we want to have a positive impact on the club as a whole. This project seeks to bring some legitimacy to the club's operations, improving the general quality of life of members and hopefully attracting more interest in the club, both from normal members and, additionally interested software members.
2. Another external goal would be to have a highly organized and streamlined development process throughout the entire project. With the use of all the GitHub features, the team will lock into an iterative development where all members' input and feedback can be properly assessed and organized to further document our work. The team wants this project to showcase our abilities to collaborate and develop a large-scale project smoothly.

Attendance

The section describes the rules and expectations outlined for team member attendance.

Expectations

Our team expects near-full attendance at all scheduled meetings, along with the required work for the meetings done beforehand. Members should be on time to scheduled meetings, and justification for leaving early must be discussed with the other members in advance. While full attendance is the standard, there will be room for members to make notice that they will be absent at a future meeting, with the added task of supplying required information before the meeting and catching up on meeting notes after. If a member misses 3 pre-scheduled meetings, there must be strong justification, and an additional meeting will be scheduled with the other members to accommodate and discuss. Approaching deliverables, meeting attendance becomes more vital and therefore after a single missed scheduled meeting, absences must be replaced with a supporting meeting or direct discussion with a member about the meeting contents. For the scheduled meetings, in-person attendance is generally expected, but there will be room for virtual attendance when other members are notified beforehand.

Acceptable Excuse

For internal deadlines that the team has set, the expectation is that other members are kept on track of the progression of work and therefore made aware that a deadline may be missed before it passes. This will ensure that if necessary, other members may assist in the required work or those internal deadlines may be adjusted. For missing meetings or when missed deadlines must be justified,

acceptable excuses will range from standard academic regulations: sickness, family responsibilities, religious accommodations, and any previously communicated commitments that the team has been made aware of. For real project deadlines, there will be no acceptable excuses unless any true emergencies have occurred.

In Case of Emergency

Support for emergency situations will begin with the continuous documentation of ongoing work, allowing for simplified transfer of assignment of tasks. If an emergency occurs and a team member either will not be able to complete their work or is even unsure of their ability to do so, they must contact and inform the team as soon as possible. The rest of the team will then attempt to accommodate and support the member by redistributing their workload, ensuring the completion of work that is to the team's standard.

Accountability and Teamwork

Quality

The team will expect nothing less than the best effort of each member, seeking to create a final product that everyone is proud to be a part of. Each member is looking to not only excel in the class, but also function smoothly as a team in the process, so preparation for meetings will be vital. Just like the final product, this preparation must be held to a high standard, ensuring that complete team progress is made smoothly.

Expectations of effort may be discussed on a per-meeting basis, ensuring that all members are on the same page about the work required to achieve the desired final product. As the scope for this project is adjustable, this communication between members will be central to keeping expectations in check.

Attitude

Central to the team's success will be remaining respectful, understanding and focusing on collaboration amongst one another. A positive attitude is generally expected, but as with any project, members may become stressed, and in those situations respect must be maintained, and discussion will be had if something breaks this expectation. Each member will also be expected to contribute a fair share of the work, both within the technical development of the project and the external tasks required. Conflict resolution will be done in a calm and respectful manner, focusing on maintaining an environment that, even through conflict, remains in check. Meetings may be scheduled with the TA and/or professor if issues escalate beyond what is feasible for the team to handle.

Stay on Track

Staying on track will be rooted in organization and communication among members, primarily utilizing GitHub task assignment features. Beyond that, team members are often attending classes together, and capstone progress is always discussed, so there is a constant general understanding of how the project develops. The weekly meetings will then formalize these updates, ensuring that all progress is documented. Expectations of members' standards as discussed above have all been a general priority of all the individual members even prior to forming this team, so adherence to these expectations is rooted in the group's interest. If a member does happen to miss a meeting or an internal deadline, a coffee or treat should be bought for the other members during the next meeting. For more important formal deadlines, any issues that arise from absences or missed submissions will result in discussion with the TA and/or professor.

Team Building

The team will build cohesion through the attendance of Troitsky events and meetings, as the team's members are introduced as formal members of the club. These events will provide a more casual place to build a team bond, while also allowing everyone the opportunity to engage with the eventual users of the platform, gaining a deeper understanding of the club itself.

Decision Making

Decisions in the group will be made democratically through a vote, with decisions involving several parameters allowing for a ranking of voting options. All split decision handling will begin with more discussion of the topic, where those with more opinion on the subject may state their reasoning and appeal to the others. In the case where it remains a tied vote, the supervisor will be consulted and will ultimately decide.